2026 全国大学生计算机系统能力大赛

编译系统设计挑战赛（华为毕昇杯）赛题技术方案

《仓颉数据竞争静态检测》

详情访问大赛官网：https://compiler.educg.net

一、赛题综述

第 1 条 《仓颉数据竞争静态检测》赛题（以下简称“本赛题”）要求参赛队实现一个静态程序分析工具识别仓颉代码中潜在的数据竞争问题。基于静态程序分析的数据竞争检查技术，可以在程序执行前，通过程序结构和语义，有效识别程序中潜在的数据竞争问题，例如：Meta 公司开发的 RacerD[1]已经应用于 Android 工程级代码的数据竞争检查，累计已经帮助 Meta 工程师识别出 2500+问题。

第 2 条 参赛队仅限于在仓颉高级中间语言 CHIR 上开发仓颉数据竞争静态检测工具，工具的实现策略包括但不限于传统程序分析技术，或者使用大模型能力加速程序分析或对传统程序分析的结果进行补充。当前开源的仓颉编译器及工具链未提供数据竞争静态检测能力，参赛队需要为开源的仓颉编译器或工具链新增数据竞争检测能力。

第 3 条 仓颉程序数据竞争的定义参见附件《仓颉编程语言规约》“并发”章节中“数据竞争 Data Race”的描述。

第 4 条 除本技术方案特别要求、规定和禁止事项外，各参赛队可在给定的参考实现基础上，自行决定所采用的优化技术等细节。

二、初赛内容与评分标准

第 5 条 比赛内容。针对仓颉编程语言代码，定义静态分析工具识别仓颉代码中潜在的数据竞争。

1. 工具的输入是一个表示仓颉工程根目录地址的字符串，目录地址下的仓颉工程满足可以被仓颉工程管理工具 cjpm 编译构建，数据竞争静态检测工具对输入的仓颉工程中的全部代码进行分析。

2. 所有作为输入的仓颉程序均基于仓颉 LTS1.0.0 版本（参见附件）中的语法和仓颉 std 标准库中提供的 API 实现。作为输入的仓颉程序可能是一个编译后可执行的程序，即：包含 main 函数作为执行入口；也可能是一个仓颉 package，支持被其他仓颉程序通过仓颉 import 语句导入并调用。
3. 数据竞争静态分析工具在确保输入输出满足赛题要求的前提下，在仓颉编译器中，增加对编译生成的中间语言 CHIR 代码做数据竞争静态检查，在所输入的仓颉工程的编译阶段识别数据竞争。
4. 允许参赛队使用大模型能力辅助或直接对程序做数据竞争静态检查，调用大模型能力的方式是本地部署还是线上调用本赛题不做限制，但请务必在所提交作品的文档中明确说明使用方式。
5. 数据竞争检测工具的输出结果是一个 .txt 文件，文件的每一行为一条识别出的数据竞争，每条识别出来的数据竞争输出格式如下：
   - 对于用例内部并发执行逻辑导致的数据竞争，输出格式为：

     (RaceType ， ((filePath1, fileName1, spawnLine1), (filePath1’, fileName1’, raceLine1’)), ((filePath2, fileName2, spawnLine2), (filePath2’, fileName2’, raceLine2’)))

     其中，RaceType 为数据竞争类型，取值为[“RW”, “WW”]。输出结果表示 filePath1 文件目录下 fileName1 文件中第 spawnLine1 行的 spawn 语句创建的线程可能与 filePath2 文件目录下 fileName2 文件中第 spawnLine2 行的 spawn 语句创建的线程产生数据竞争，产生数据竞争的地方为 filePath1’目录下的 fileName1’文件中第 raceLine1’行的语句与 filePath2’目录下的 fileName2’文件中第 raceLine2’行的语句。
   - 对于外部程序调用用例中暴露的 public 接口导致的数据竞争，数据格式为：

     (RaceType ， ((filePath1, fileName1, declareLine1), (filePath1’, fileName1’, raceLine1’)), ((filePath2, fileName2, declareLine2), (filePath2’, fileName2’, raceLine2’)))

     表示外部程序并发访问 filePath1 文件目录下 fileName1 文件中第

declareLine1 行声明的变量/方法，与 filePath2 文件目录下 fileName2 文件中第 declareLine2 行声明的变量/方法，可能产生数据竞争，产生数据竞争的地方为 filePath1’目录下的 fileName1’文件中第 raceLine1’行的语句与 filePath2’目录下的 fileName2’文件中第 raceLine2’行的语句，其中数据竞争类型为 RaceType。

第 6 条 功能测试。仓颉数据竞争静态分析工具识别出仓颉源码中真实数据竞争越多，漏报的真实数据竞争越少，得分越高。用于评分的用例有多组，每组用例分别作为参赛队所开发的数据竞争静态分析工具的输入，参赛队开发的工具需要对每组用例分别给出分析结果，每组测试用例最高得 2 分。如果测试用例本身是无数据竞争的，输出 0 条数据竞争得 2 分，否则不得分；如果用例存在数据竞争，则得分遵循如下公式：

$$\text{得分} = \frac{TP}{TP+FP} * 1.0 + \frac{TP}{TP+FN} * 1.0$$

其中：

- TP (True Positive)：正确报告的真实数据竞争。
- FP (False Positive)：误报的非数据竞争。
- FN (False Negative)：漏报的真实数据竞争。
- TP + FP：工具报告总数。
- TP + FN：真实数据竞争的总数。

对于每组测试用例，数据竞争静态检测工具需在 5 分钟内给出输出，否则不得分，若给出输出的时间超过 3 分钟但小于 5 分钟，则扣 0.8 分。最终得分（满分 100 分）为所有测试用例得分之和。

第 7 条 作为测试评分的仓颉源码共 50 组，包含：拥有 main 函数入口的程序 40 组，以及允许被其他仓颉模块调用的仓颉包 10 组。每组得分规则参考第 6 条。大赛会公开 50 组测试用例中的 5 组，以方便参赛队理解赛题并调试所开发的工具。所有测试用例的程序规模控制在 5k 行仓颉代码以内，包括程序中所调用的仓颉标准库代码。

三、决赛内容和评分标准

第 8 条 决赛包括两个阶段，第一阶段是继续优化作品，赛题要求和初赛保持一致。第二阶段是现场答辩。现场答辩主要考察参赛队的团队协作及提交作品的代码完成度、方案设计文档、技术创新性及代码规范性等因素以及现场答辩效果，具体比赛要求和评分标准将在决赛阶段另行公布。

第 9 条 决赛额外增加 10 组不公开的测试用例用于评分。每组测试用例 3 分，其中功能性得分 2 分，性能性加分 1 分，总计 30 分。其中，功能性得分参考第 6 条。对于每组测试用例，如果存在多个参赛队的结果完全正确，即：功能性得分为 2，则根据工具输出结果的时间对参赛队进行排序，排名前 4 的队可分别获得 1 分、0.75 分、0.5 分、0.25 分的得分。

第 10 条 决赛成绩按如下权重计算：

1. 客观分（与初赛一致）：30%。
2. 方案设计文档得分：20%。
3. 参赛队团队协作及现场答辩：50%。

四、参赛作品提交

第 11 条 各参赛队在初赛和决赛阶段需要在大赛的竞赛平台提交如下内容：

1. 仓颉并发数据竞争静态检测工具的实现源码。参赛队应保证所提交源码可以在竞赛平台上运行。大赛提供参考测试工具/方法，参赛队可用于辅助调试，最终测试结果以组委会复核结果为准。
2. 相关设计文档（PDF 格式），内容包括但不限于对功能、性能及创新性的说明、相关测试结果及与类似项目的对比分析、遇到的瓶颈问题和解决方法等。

第 12 条 参赛队在初赛阶段需要提供时长不超过 5 分钟的作品介绍视频（MP4 格式）。

第 13 条 允许参赛队部分代码参考或使用遵循开源协议的编译系统或软件模块。如果需要使用第三方 IP 或者借鉴他人的部分源码，必须在设计文档和源代码的头部予以明确说明，并确保内容符合相关法律法规和开源协议

之规定。

第 14 条 参赛队必须严守学术诚信。一经发现存在代码抄袭或使用他人代码不做说明等学术不端行为，修改的代码重复率在 20%以上，取消参赛队的参赛资格。

第 15 条 参赛队在赛后应按大赛要求公开代码和文档。

五、竞赛平台与测试程序

第 16 条 本赛题提供的竞赛平台和测试程序包括：

1. 代码托管平台，支持各参赛队的群体协作与版本控制。
2. 竞赛评测系统，根据参赛队的申请从代码托管平台获取指定版本，生成编译系统，并加载基准测试程序，自动进行功能和性能测试。
3. 测试平台：鲲鹏 920ARM 服务器。

六、软硬件系统规范

第 17 条 大赛指定的编译环境用于参赛队提交的源码，参数如下：

1. CPU：鲲鹏 920 ARM。
2. 操作系统：openEuler。

七、大赛网站

第 18 条 大赛官网：https://compiler.educg.net。

第 19 条 大赛官网提供多种软件开发工具及设计资料，包括但不限于下列内容：

1. 竞赛平台（代码托管平台、竞赛测试系统）使用文档。
2. 仓颉文档。
3. 仓颉社区。

八、其他

第 20 条 本赛题最终解释权归全国大学生计算机系统能力大赛编译系统设计赛技术委员会。

全国大学生计算机系统能力大赛组委会

编译系统设计赛技术委员会

2025 年 11 月

参考文献

[1] Sam Blackshear, Nikos Gorogiannis, Peter W. O'Hearn, Ilya Sergey: RacerD: Compositional Static Race Detection. In OOPSLA(2018).

[2] Vineet Kahlon, Nishant Shinha, Erik Kruus, Yun Zhang: Static Data Race Detection for Concurrent Programs with Asynchronous Calls. In ESEC/FSE(2009)

[3] Cangjie source code：https://gitcode.com/Cangjie

[4] Cangjie LTS1.0.0 Docs：https://cangjie-lang.cn/docs?url=/1.0.0/index.html

[5] 仓颉编程语言规约：
https://cangjie-lang.cn/docs?url=%2F0.53.18%2FSpec%2Fsource_zh_cn%2FChapter_15_Concurrency%28zh%29.html